差速小车系统架构与五人分工

1　系统图

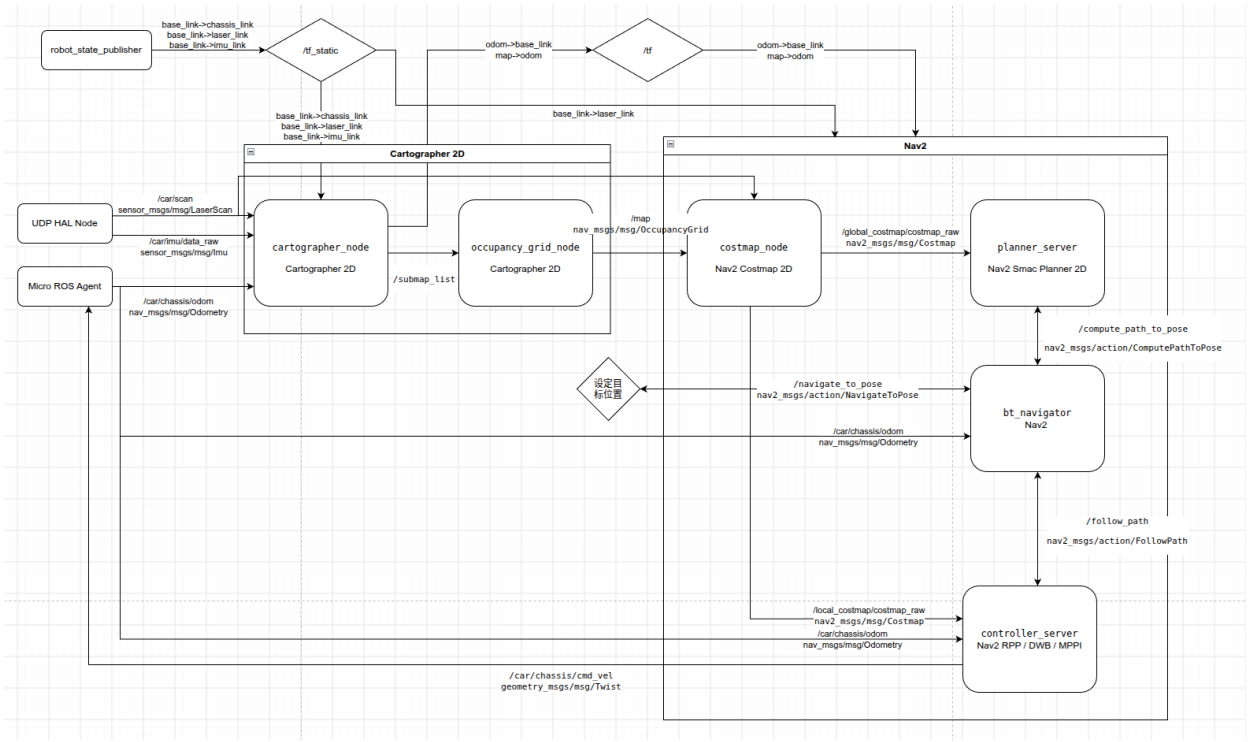


图 1: 工程流程与消息拓扑

系统主链路。全局和局部代价地图分别由规划服务器、控制服务器管理，插件在进程内读取代价地图。接口消息类型、参考频率和发布责任见下表。

1.1 硬件拓扑

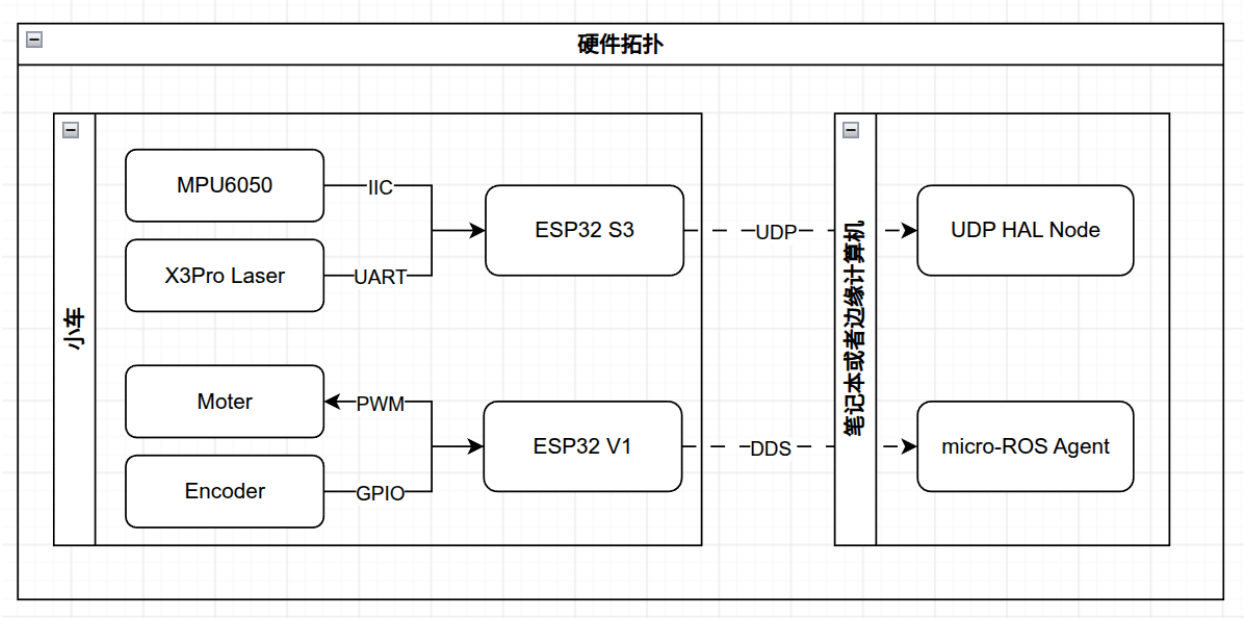


图 2: 小车与上位机硬件连接拓扑

MPU6050 与 X3 Pro 分别通过 I^2C 和 UART 接入 ESP32-S3，再经 UDP 交给上位机 HAL；电机与编码器接入 ESP32 V1，底盘通过 micro-ROS Agent 连接 ROS 2。

1.2 运行结构与维护边界

ESP32-S3 采集 X3 Pro 激光雷达和 MPU6050 数据，通过统一 UDP 协议交给上位机 UDP HAL。ESP32 V1 完成底盘闭环控制，通过 micro-ROS Agent 与 ROS 2 通信。Agent 负责传输桥接，里程计由底盘程序计算。

上位机的一个 `robot_state_publisher` 从统一 URDF/Xacro 发布全部固定安装变换。Cartographer 读取激光、IMU 和底盘里程计，输出动态定位 TF；`occupancy_grid_node` 生成 `/map`。本版定位来源统一采用 Cartographer，Nav2 启动导航部分，不同时启用另一套定位节点竞争同一段 TF。[1][2]

Nav2 采用 Smac Planner 2D 进行全局规划，当前工作区使用 MPPI 和旋转对齐控制链路，并保留 RPP 作为可替换方案；按需要加入 `velocity_smoother`。第 5 人负责把速度平滑、限速及底盘最终输出接口整合好，第 4 人负责把同一套参数和启动流程从仿真迁移到实车。DWB 或 RPP 作为后续替换选项，不要求首版同时调通多种控制插件。

1.3 接口频率与消息约定

接口（参考频率）	类型	提供方与使用方
`/car/scan` 10 Hz	`sensor_msgs/msg/LaserScan`	UDP HAL 发布；Cartographer 与已启用雷达输入的代价地图图层使用
`/car/imu/data_raw` 50 Hz	`sensor_msgs/msg/Imu`	UDP HAL 统一发布；IMU 模块和标定由第 3 人负责，第 2 人负责合入公共通信；Cartographer 使用

接口（参考频率）	类型	提供方与使用方
/car/chassis/odom 20 Hz	nav_msgs/msg/Odometry	底盘经 Agent 发布；Cartographer 使用位姿序列，Nav2 控制器和导航器使用速度反馈
/map 1 Hz	nav_msgs/msg/OccupancyGrid	occupancy_grid_node 发布；全局代价地图静态层使用
/tf_static 静态/按需	tf2_msgs/msg/TFMessage	上位机唯一的 robot_state_publisher 发布固定安装变换
/tf 20 Hz	tf2_msgs/msg/TFMessage	Cartographer 发布两段定位变换；各使用方通过 tf2 查询
/navigate_to_pose 按需	nav2_msgs/action/NavigateToPose	上层任务节点或 RViz2 作为客户端；bt_navigator 作为服务端
/compute_path_to_pose 按需	nav2_msgs/action/ComputePathToPose	bt_navigator 请求；planner_server 返回包含 Path 的规划结果
/follow_path 按需/反馈 10 Hz	nav2_msgs/action/FollowPath	bt_navigator 提交包含 Path 的请求；controller_server 执行并反馈状态
/car/chassis/cmd_vel 20 Hz	geometry_msgs/msg/Twist	Nav2 最终速度输出经重映射到此接口；ESP32 V1 订阅
/global_costmap/costmap_raw 发布 1 Hz	nav2_msgs/msg/Costmap	全局代价地图对外话题；planner_server 在进程内读取同一代价地图
/local_costmap/costmap_raw 发布 5 Hz	nav2_msgs/msg/Costmap	局部代价地图对外话题；controller_server 在进程内读取同一代价地图
/car/reset_odometry 按需	std_srvs/srv/Trigger	项目自定义管理接口，由第 3 人实现，第 4 人统筹运行条件，第 5 人协调导航状态

第 5 人应显式配置 Nav2 速度链路的消息类型。本版统一使用 geometry_msgs/msg/Twist，主要字段是 linear.x 和 angular.z；若所选发行版默认使用 TwistStamped，应统一配置或在边界转换，不能让同名接口混用两种类型。ROS 2 发行版、依赖分支、IP 端口和 QoS 在首次联调前统一记录。

/global_costmap/costmap_raw 和 /local_costmap/costmap_raw 的类型是 nav2_msgs/msg/Costmap，用于对外访问；标准 Nav2 的原生规划和控制插件不依靠订阅这两个话题接入代价地图。对应的 costmap 可视化话题使用 nav_msgs/msg/OccupancyGrid。[5]

1.4 坐标变换与时间同步

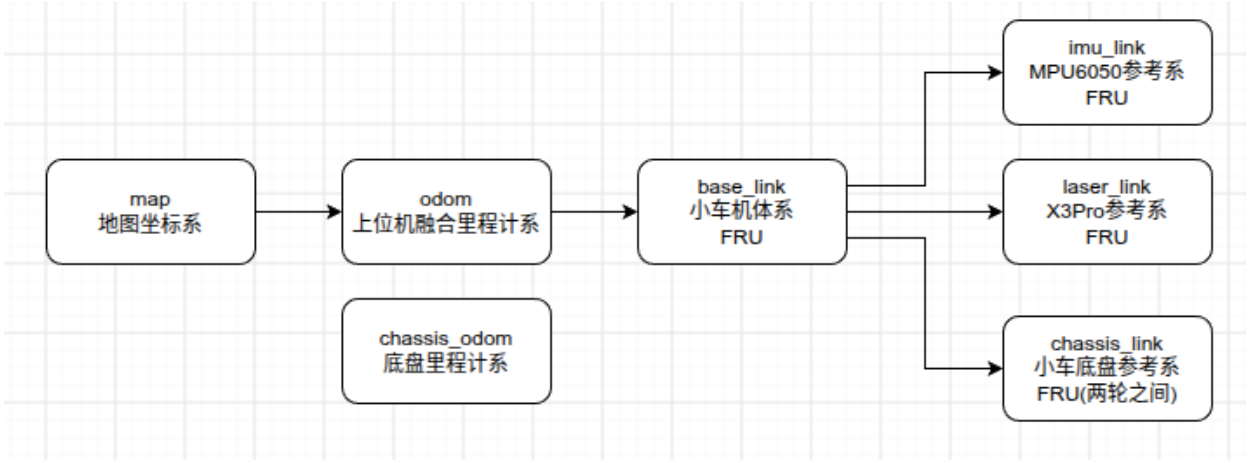


图 3: 统一坐标变换关系与发布责任

固定安装关系统一写入一份 URDF/Xacro，由上位机唯一的 `robot_state_publisher` 发布；底盘和 HAL 不重复发布安装 TF。Cartographer 负责 `map`→`odom`→`base_link` 两段动态 TF。原始里程计消息可以使用 `chassis_odom` 作为参考系，但不因该消息再广播一条 `chassis_odom`→`base_link` TF。

micro-ROS 链路要单独处理时间同步：MCU 采样时钟、Agent 主机时钟和 ROS 时钟可能存在偏移，不能把上位机收包时刻直接当成传感器采样时刻。启动和重连时执行 micro-ROS 时间同步，记录同步是否成功及估计偏移；UDP 包保留采集时间、序号和重启标识，HAL 按统一 ROS 时间发布。同步失败时要有明确诊断，并在验收记录中说明采用的降级时间基准，不能让时间戳倒退或跨传感器无规律跳变。

2 人员任务与验收

人员	主要负责内容	交接对象
第 1 人史子墨	底盘驱动、编码器、差速运动学、闭环控制、micro-ROS	向第 3 人提供原始里程计，向第 4 人提供仿真/实车对比数据，向第 5 人提供运动能力与反馈
第 2 人张子安琪	雷达、ESP32-S3 公共程序、UDP 协议、上位机 HAL	向第 3 人提供激光和 IMU 公共通信链路，向第 4、5 人提供可回放数据
第 3 人罗君豪	IMU、标定、统一机器人模型、静态 TF、Cartographer 及归零管理	向第 4 人提供可迁移的定位配置，向第 5 人提供地图、定位 TF 和外参
第 4 人张殷瑞	项目统筹、接口冻结、Gazebo 仿真、sim-to-real 迁移、跨模块联调	汇总各模块交付，`sim_to_real`
第 5 人辛祚印	两套代价地图、全局规划、局部控制、行为树、目标下发及启动集成	向第 1 人输出速度指令，牵头端到端导航验收

2.1 第 1 人史子墨：底盘运动控制

1. 工作平台

ESP32 V1、双轮差速底盘、电机驱动和左右轮编码器；固件使用 ESP-IDF 与 C 语言，通信使用 micro-ROS 的 C 接口。上位机使用 Ubuntu、ROS 2 Jazzy 和 micro-ROS Agent，提供串口烧录、日志和话题检查环境。

2. 工程简述

完成 ESP32 V1 上的电机驱动、编码器读取、左右轮速度闭环和差速解算；接入 micro-ROS Wi-Fi 通信，提供可供 SLAM 和 Nav2 使用的原始 Odometry。实现指令限幅、无效指令处理和通信超时停车。

3. 输入接口

- `/car/chassis/cmd_vel` 的 Twist 指令，使用车体前进速度和绕 z 轴角速度。
- 左右轮编码器计数、实际轮径、有效轮距、电机驱动引脚和允许转速。

4. 输出接口

- `/car/chassis/odom`，包含有效位姿、线速度、角速度和时间戳；不发布车体定位 TF。

5. 交付项

- 可编译烧录的底盘固件工程、接线表和 micro-ROS Agent 启动说明。
- 轮径、轮距、编码器换算与 PID 参数表，以及方向、速度闭环和里程计测试记录。

6. 大致实现路线

MCU 定时读取编码器并计算轮速，将车体速度指令换算为左右轮目标速度，再通过 PID 控制 PWM 和方向。轮速闭环在 MCU 本地执行，不依赖 Wi-Fi 逐次返回控制结果。

根据实测轮速积分原始里程计，同时填充 Odometry 中的位姿和速度；明确计数方向、时间间隔和协方差。

7. 交付测试项

检查项	操作与通过条件	留存证据
方向和解算	分别给前进、后退、左转、右转指令，轮子方向与车体响应均正确	操作视频与指令记录
速度闭环	选择两个高于死区的稳定轮速，去掉启动前 1 s 后，编码器反馈均值与目标差不超过 10%	左右轮目标与反馈曲线
里程计	正常网络条件下连续运行 5 min，平均频率不低于设定值 90%，时间不倒退；直行 1 m 累计距离误差建议不超过 10%	rosbag、频率统计及卷尺记录

2.2 第 2 人张子安琪：雷达与统一通信

1. 工作平台

ESP32-S3、X3 Pro 二维激光雷达、Wi-Fi 网络与上位机。MCU 公共程序使用 ESP-IDF 和 C 语言，上位机 UDP HAL 使用 ROS 2 节点接入，使用串口工具、UDP 收发工具和 RViz2 检查数据。

2. 工程简述

完成 X3 Pro 数据解析、ESP32-S3 公共采集程序、Wi-Fi UDP 通信和上位机 UDP HAL。统一承接激光与 IMU 数据通路；IMU 采集和标定模块由第 3 人提供，第 2 人负责合入同一套通信框架。

3. 输入接口

- 雷达串口数据、IMU 数据。

4. 输出接口

- `/car/scan`；`/car/imu/data_raw`。

5. 交付项

- ESP32-S3 公共固件、雷达解析模块和 IMU 模块接入说明。
- UDP 协议文档、上位机 HAL 节点及其启动配置。
- 雷达/IMU 数据样例、通信统计和本节测试记录。

6. 大致实现路线

写一套 UDP 协议, 以及上位机 HAL 节点以发布 imu 及激光雷达数据

7. 交付测试项

检查项	操作与通过条件	留存证据
扫描方向	RViz 中前、左、右障碍与实物对应；一圈扫描的角度、点数及增量关系一致	RViz 截图与摆放照片
测距	对准平整墙面，在 0.5 m、1 m 和 2 m 各采样；中心附近有效点的距离中位数与卷尺读数差建议不超过 0.05 m	原始扫描与测量表
稳定通信	同一 AP、设备与上位机距离不超过 3 m，连续运行 10 min，无进程崩溃；平均扫描频率不低于设定值 90%	rosbag 与协议计数日志
异常数据	注入丢包、乱序和截断包，能够丢弃无效帧并计数；后续完整帧到达后自动恢复，无需重启 HAL	注入脚本与日志
交接完整性	第 3 人可按说明直接订阅扫描和 IMU 并接入 Cartographer；第 4 人可按说明复现仿真和实车接口；消息单位、frame_id、时基与约定一致	接口检查记录

2.3 第 3 人罗君豪：IMU、标定与融合建图定位

1. 工作平台

MPU6050 与 ESP32-S3 公共固件；上位机使用 ROS 2 Jazzy 与 Cartographer 2D，由 `robot_state_publisher` 提供安装 TF。IMU 模块使用 C 语言接入第 2 人的固件，机器人模型使用 URDF/Xacro，Cartographer 参数使用 Lua，数据验证使用 RViz2、ROS 工具和 rosbag。

2. 工程简述

完成 MPU6050 采集与标定，维护统一 URDF/Xacro 和所有固定安装外参，运行上位机唯一的静态 TF 发布节点。接入 Cartographer 的激光、IMU 和底盘里程计，完成融合定位、栅格建图及地图保存。实现项目归零管理接口，并向第 4 人和第 5 人说明定位连续性与导航运行条件。

3. 输入接口

- 第 2 人提供的 `/car/scan` 和`/car/imu/data_raw`。
- 第 1 人提供的 `/car/chassis/odom`。

4. 输出接口

- `/map` 和两段动态定位 TF。

5. 交付项

- MPU6050 采集与标定模块、标定结果和统一 URDF/Xacro。
- 固定安装 TF 启动配置、Cartographer Lua 配置、状态文件和导航栅格地图。
- IMU、三路融合、建图定位及归零服务测试记录。

6. 大致实现路线

完成静止零偏标定、坐标轴与单位核对

URDF 以 base_link 为根，统一维护 chassis_link、laser_link 和 imu_link 的固定关系。上位机一个 robot_state_publisher 负责 `/tf_static`；底盘和 HAL 不重复发布安装 TF。固定 joint 启动即可发布，不需要伪造轮关节状态。[1]

用 Cartographer 进行小车底盘里程计,imu, 激光雷达融合.

使用 occupancy_grid_node 生成 `/map`，同时保存 Cartographer 状态和供导航使用的栅格地图。

`/car/reset_odometry` 是需另行适配的管理服务，请求无参数，将当前停车位置设为局部里程计零点、当前车头方向设为零航向，响应通过 `success` 和 `message` 报告结果；

7. 交付测试项

检查项	操作与通过条件	留存证据
IMU 标定	静止录制 60 s；校正后各轴角速度均值绝对值建议不超过 0.02 rad/s，加速度模长处于 9.81 ± 0.50 m/s^2	标定前后数据与曲线

检查项	操作与通过条件	留存证据
IMU 接口	平均频率不低于 50 Hz 目标的 90%；角速度为 rad/s、加速度为 m/s^2；转动车体时轴向和符号正确	rosbag 与字段核对表
静态 TF	`/tf_static` 只有一个项目发布节点；三个固定关系与安装测量一致；晚启动的监听者也能查到变换	发布者清单、TF 树和查询记录
三路融合	激光、IMU、里程计均被接收，无持续缺失变换或时间倒退错误；停止任一输入时有明确诊断	运行日志与数据回放
建图与定位	在标记起终点的室内闭环路线行驶，总路程约 10 m；返回后地图中的起终点位姿差建议不超过 0.30 m 和 10 度；重复 3 次并记录全部结果	地图、rosbag、现场标记和误差表
连续局部 TF	回环前后分别绘制两段 TF；map 到 odom 允许修正，odom 到 base_link 应保持局部运动连续，不因地图优化直接重置	对齐时间轴的 TF 曲线

归零服务单独验收：至少包含一次满足条件的成功归零，以及行驶中、导航未停止、数据无效等拒绝场景。成功后局部位置与航向在约定容差内归零，TF 仍只有原定发布者；若同时提供 `/car/odom`，其位姿与 TF 一致。仅返回拒绝的占位实现不计作归零功能通过。运动条件、Cartographer 运行状态处理、是否保留地图及输入参考基准处理，应在开发前写成明确方案并冻结；未验证成功流程前不在实车导航中启用。

2.4 第 5 人辛祚印：Nav2 导航与系统集成

1. 工作平台

上位机 ROS 2 Jazzy 与 Nav2，使用 Smac Planner 2D、MPPI、Rotation Shim、速度平滑和碰撞监测组件。参数使用 YAML，行为树使用 XML，目标下发与验证使用 ROS 工具和 Python；与第 4 人的 Gazebo 和实车环境分别联调。

2. 工程简述

完成 Nav2 全局规划、局部控制、行为树和目标接口，维护导航参数与总启动文件。负责两套代价地图及其图层配置、速度约束、路径更新处理、取消任务和故障反馈，牵头完成从目标下发到底盘运动的整体验收。

3. 输入接口

- 第 3 人提供的 `/map`、动态定位 TF、统一机器人模型和静态 TF 启动配置。
- 第 2 人提供的 `/car/scan`，以及第 1 人提供的 `/car/chassis/odom` 和实测运动限制。
- 上层任务节点或 RViz2 经 `/navigate_to_pose` 提交的目标位姿，以及已冻结的验收场地和目标点。

4. 输出接口

- `/navigate_to_pose`、`/compute_path_to_pose`、`/follow_path` 三个 Action 接口及反馈、取消和结果处理。
- 最终 `/car/chassis/cmd_vel`；其消息类型、速度和加速度限制与底盘一致。

5. 交付项

- Nav2 参数 YAML、行为树 XML、导航启动文件及目标下发/取消操作说明。
- 代价地图、规划器、控制器和速度输出链路配置。
- 到点导航、临时障碍、路径更新、取消和故障测试记录。

6. 大致实现路线

planner_server 管理 map 系下的 global_costmap，并加载 Smac Planner 2D；controller_server 管理 odom 系下的 local_costmap，并加载 MPPI 与旋转对齐控制链路。全局地图叠加静态、障碍和膨胀层；局部采用围绕车体的滚动窗口，优先使用实时激光与膨胀层。两套地图的范围、分辨率、车身轮廓及频率根据真实底盘配置。

bt_navigator 接收 NavigateToPose 目标，调用 ComputePathToPose 取得 Path，再通过 FollowPath 交给控制器。bt_navigator 和 controller_server 均将 `odom_topic` 指向 `/car/chassis/odom`。服务器内部使用各自代价地图，costmap_raw 只作为对外接口。[5][6]

首版先完成旋转对齐、MPPI 跟踪与前向碰撞检查，再配合速度平滑、路径守卫和全局重规划处理新障碍。新 Path 到来时应保证速度变化符合配置约束；接续不可行时减速或停车，不锁定已经出现障碍的旧路径。局部控制器不是全局绕行规划器；需要替换 RPP、DWB 或 MPPI 时，必须重新冻结参数并验收。[7]

7. 交付测试项

检查项	操作与通过条件	留存证据
接口与启动	按一份操作说明启动；三条 Action 可用，TF、地图和传感器就绪后进入导航；重复冷启动 3 次	启动日志与 Action 清单
两套代价地图	全局覆盖规划区域，局部窗口随车移动；放置与移走箱子后能标记、清除障碍；变化到地图更新建议不超过 0.50 s	RViz 录像及时间记录
到点导航	预设 5 个可达目标，各测试 2 次；建议成功不少于 9 次，全部无碰撞；成功时位置误差不超过 0.20 m、朝向误差不超过 10 度	10 次结果表与实测记录
近处新障碍	行驶途中加入可检测障碍，控制器及时减速或停车；绕行通道存在时通过重规划或所选控制器继续，受阻时明确报告	局部地图、速度曲线和录像
路径更新	在有两条可行路线的场景触发重规划；线速度、角速度和加减速度均不超冻结限制，无持续左右摆动或追逐身后点	新旧 Path 与速度时间曲线
取消和故障	取消目标、目标不可达、停止扫描、停止里程计、停止定位 TF，逐项测试；任务返回明确状态，车体停车且不继续执行陈旧任务	Action 结果、日志及停车时间

2.5 当前工作区代码规范

本节以 `/home/doc049/dev/smartcar_ws` 当前目录为准，适用于后续新增节点、配置和验证脚本。代码、配置、启动文件和测试都要能在干净的工作区中按 README 的命令复现；临时调参可以放在个人记录中，但通过验收的参数必须回写到包内配置并说明原因。

1. **包与目录**。ROS 2 包统一使用 `smartcar_<domain>` 命名。Python 节点放在同名 Python 子包中，入口在 `setup.py` 的 `console_scripts`；C++ 节点的头文件放在 `include/<package>/`，

实现放在 src/，通过 CMakeLists.txt 安装。启动文件放在 launch/，参数放在 config/，URDF/Xacro 放在 urdf/，行为树放在 behavior_trees/。

2. **命名与接口**。文件、函数、参数、话题和服务使用小写蛇形命名；类名使用首字母大写的单词组合；节点名和 frame 名不带开头斜杠。公共接口以 /car/scan、/car/imu/data_raw、/car/chassis/odom、/car/chassis/cmd_vel 和 /map 为准，变更必须同步更新启动文件、参数、文档和验证脚本。
3. **消息与 TF**。当前 Nav2 参数显式设置 enable_stamped_cmd_vel:false，速度链路使用 geometry_msgs/msg/Twist。固定安装 TF 只由统一的 robot_state_publisher 发布，动态定位 TF 只由 Cartographer 发布。当前工作区的 Cartographer Lua 使用 odom_frame="chassis_odom"，运行时链路是 map->chassis_odom->base_link；若接口冻结要改为 map->odom->base_link，必须一次性修改配置、测试和所有文档，不能混用两套名称。
4. **参数与 QoS**。节点启动时声明参数并给出安全默认值；运行时可调参数集中在 YAML 或 Lua，不把关键常数散落在回调函数中。传感器使用 SensorData QoS，静态地图或需要晚加入者收到的代价地图使用与用途匹配的持久性和队列深度；QoS 变更要在接口冻结表中记录。
5. **Python 与 C++ 实现**。Python 文件包含模块说明，使用四空格缩进、snake_case 和显式的 main()；节点退出时释放实体并关闭 rclpy。C++ 使用 RAII、const 和明确的智能指针类型，回调只做必要的数据转换，日志使用 RCLCPP_*，不得用标准输出替代节点日志。新增订阅、发布和服务时同时检查空指针、时间戳、消息单位和异常输入。
6. **启动与仿真**。总启动流程通过 smartcar_bringup 组织，仿真相关内容放在 smartcar_gazebo、smartcar_sim_io 和 smartcar_description；Gazebo 私有里程计和 TF 不得绕过 Cartographer 成为第二套动态定位。仿真和实车共用公开话题、frame 和参数名，差异只通过启动参数或明确的适配节点表达。
7. **验证与测试**。修改节点或配置后至少执行 colconbuild--symlink-install 和 python3scripts/validate_system.py，再按改动范围运行验证脚本或 colcontest。运行时留存话题频率、Action 结果、TF 查询和必要的 rosbag；测试脚本必须报告明确的通过/失败条件，不能只打印一段无结论的日志。
8. **版本与提交**。提交前删除个人临时文件，禁止把 build/、install/、log/、缓存、生成地图和未审查的 rosbag 加入版本库。每次接口或参数变化在提交说明中写明影响的话题、TF、消息类型、参考频率和验收命令；修复问题时保留失败证据和复测结果。

2.6 官方接口依据

- 1 Robot State Publisher 固定关节与静态 TF
- 2 Cartographer 输入输出与栅格地图节点
- 3 Cartographer ROS 2 传感器适配与跟踪坐标系要求
- 4 sensor_msgs Imu 单位与无姿态估计的标记
- 5 Nav2 规划服务器的代价地图管理
- 6 NavigateToPose 接口 与 FollowPath 接口
- 7 Regulated Pure Pursuit 控制器